

Progettazione di Programmi Paralleli

Introduzione al Parallel Computing

Il **Parallel Computing** (calcolo parallelo) nasce dall'esigenza di abbattere i tempi di esecuzione per problemi computazionalmente complessi. La strategia di base consiste nello scomporre un problema di grandi dimensioni in sottoproblemi più piccoli, i quali possono essere risolti simultaneamente da più unità di calcolo per ottenere il risultato in un tempo minore.

Per parallelizzare un algoritmo, il primo passo è capire come dividerlo. Esistono due macro-strategie di decomposizione:

- **Decomposizione di dominio (Data Parallel):** si applica in presenza di grandi dataset strutturati in modo regolare, come array o matrici. Il dataset viene "affettato" in sottodomini più piccoli. Ogni task esegue esattamente le stesse istruzioni, ma su porzioni di dati differenti. Questa suddivisione può avvenire in una o più dimensioni (1D, 2D) e con logiche diverse, ad esempio a blocchi (block) o in modo ciclico (cyclic).
- **Decomposizione funzionale (Task Parallel):** invece di dividere i dati, si divide il lavoro. Il problema viene frammentato in base alle funzioni o alle elaborazioni indipendenti che devono essere svolte. È molto scalabile se ci sono molte operazioni indipendenti, ma è vantaggiosa solo quando queste funzioni sono sufficientemente complesse da giustificare lo sforzo di gestione dei task.

Interazione tra task: comunicazione e sincronizzazione

In un sistema parallelo, i task raramente lavorano nel vuoto; spesso hanno bisogno di coordinarsi. La dinamica dell'applicazione determina il grado di accoppiamento (coupling) tra i processi:

- **Embarassingly Parallel:** sono i problemi ideali. La decomposizione del dominio non richiede alcuna comunicazione tra i task. Un classico esempio è l'inversione dei colori dei pixel di un'immagine da bianco a nero: ogni pixel può essere elaborato indipendentemente.
- **Loosely Coupled:** i task comunicano molto raramente. Lo fanno tipicamente solo all'inizio (per ricevere i dati o i compiti) e alla fine (per restituire i risultati).
- **Tightly Coupled:** richiedono continue interazioni e scambi di dati a ogni iterazione. Ad esempio, nella simulazione della diffusione del calore, il valore di temperatura di una cella dipende strettamente dalle celle adiacenti gestite da altri task.

Modelli e costi di comunicazione

Nei sistemi a memoria distribuita, la comunicazione avviene principalmente in due modalità:

- **Punto-punto:** la comunicazione avviene tra soli due task specifici.
- **Collettiva:** coinvolge tutti i task dell'applicazione. Le varianti più comuni includono broadcast (uno a tutti), scatter (distribuzione), gather (raccolta) e reduction (raccolta con elaborazione, es. somma globale).

La comunicazione non è mai gratuita e influisce pesantemente sulle prestazioni. Il costo di un messaggio punto-punto dipende dalla latenza (il tempo fisico di avvio della comunicazione) e dalla larghezza di banda: $T_{mess} = T_{lat} + \frac{M}{Band}$. Per le comunicazioni collettive, un approccio ingenuo costerebbe proporzionalmente al numero di task $O(P - 1)$. Tuttavia, utilizzando logiche "divide et impera" ad albero, il costo può crollare a $O(\log_2(P))$, con un enorme risparmio di tempo nei sistemi con molti processori.

Sincronizzazione e bilanciamento del carico

A volte un task deve aspettare un altro non per scambiare dati, ma per dipendenze logiche (es. il task 1 può procedere solo se il task 2 ha finito una certa fase). Gli strumenti per gestire ciò sono:

- **Sistemi a memoria condivisa:** lock e semafori (un task si blocca in attesa che una risorsa venga sbloccata).
- **Sistemi a memoria distribuita:** scambio esplicito di messaggi d'attesa.
- **Entrambi:** barriere (barrier), in cui nessun task può procedere finché tutti non hanno raggiunto lo stesso punto.

I tempi morti in cui un task aspetta sono detti **tempi di Idle** e degradano le prestazioni. Per evitarli si usa il **Load Balancing** (bilanciamento del carico), spesso implementato tramite il modello **Master-Slave**: un "master" genera piccoli incarichi e li assegna dinamicamente a un pool di "slave" man mano che questi si liberano.

Il compromesso della granularità

In quante fette tagliamo il problema?

- **Grana fine (Fine Grain):** ottima per bilanciare il carico e ridurre l'attesa nei sincronismi, ma genera un enorme overhead (costo gestionale) per creare e far comunicare i tanti piccoli task.
- **Grana grossa (Coarse grain):** migliora il rapporto tra lavoro utile e comunicazione, ma rende molto difficile bilanciare il carico tra i processori, rischiando colli di bottiglia.

Analisi delle prestazioni: efficienza e legge di Amdahl

L'obiettivo del parallelismo è l'accelerazione, misurata dallo **Speedup**, ossia il rapporto tra il tempo di calcolo seriale e quello parallelo: $Speedup = \frac{T_{seriale}}{T_{parallelo}}$.

L'**efficienza** relaziona lo speedup al numero di processori utilizzati ($E = \frac{Speedup}{P}$); idealmente vale 1, ma nella realtà è sempre minore.

Valutare un sistema richiede di testarne la **scalabilità**, misurabile in due modi:

- **Strong Scaling:** misura l'efficienza mantenendo fissa la dimensione totale del problema mentre si aumentano i processori.
- **Weak Scaling:** misura l'efficienza mantenendo fissa la dimensione del problema *per ogni singolo processore* (se raddoppio i processori, raddoppio anche la mole di dati complessiva).

Limiti teorici: overhead e legge di Amdahl

Perché lo speedup ideale non viene mai raggiunto? A causa degli overhead (T_{oh}): tempi spesi in comunicazioni, inattività (idle) e gestione vera e propria di thread o processi.

Inoltre, il parallelismo sbatte contro un muro teorico sancito dalla legge di Amdahl. Qualsiasi programma ha una parte parallelizzabile (T_p) e una strettamente sequenziale (T_{np}). Lo speedup massimo è matematicamente limitato dalla porzione sequenziale: $S_{amdahl} = \frac{1}{Q_{np} + \frac{Q_p}{P}}$ (dove Q rappresenta le quote percentuali). Se anche usassimo infiniti processori, il tempo d'esecuzione non potrebbe mai scendere sotto il tempo necessario a eseguire la parte sequenziale del codice.

Livelli di parallelismo e paradigmi di programmazione

Esistono vari livelli per scrivere codice parallelo. Un esempio eccellente riguarda la programmazione vettoriale tramite SIMD (Single Instruction Multiple Data), dove un'operazione viene eseguita in parallelo su un intero array di elementi:

- **Automatico:** il compilatore (es. GCC) analizza i loop e li parallelizza da solo.
- **Guidato:** il programmatore aiuta il compilatore inserendo delle direttive testuali, come le istruzioni `#pragma` in OpenMP, che possono suggerire allineamenti di memoria o forzare vettorizzazioni.
- **Esplicito:** uso di "intrinsic", ovvero funzioni C native che il compilatore traduce direttamente in complesse istruzioni assembly SIMD, fornendo il massimo controllo ma richiedendo più abilità dal programmatore.

Modelli Architeturali

Per orchestrare intere macchine multiprocessore, si usano standard precisi legati a come è gestita la memoria:

1. Memoria Condivisa (Shared Memory):

- Tutti i thread o processi leggono e scrivono sulle stesse variabili. Richiede l'uso di lock e semafori per evitare collisioni di scrittura.
- Si programma con thread POSIX o più facilmente con **OpenMP**. OpenMP sfrutta il modello **Fork/Join**: un thread Master esegue una `fork` creando un team di thread paralleli per dividersi, ad esempio, le iterazioni di un loop; al termine, eseguono un `join` tornando al singolo thread sequenziale.

2. Memoria Distribuita (Message Passing):

- Non ci sono risorse in comune. Ogni processo possiede la sua memoria isolata. L'unica forma di interazione è spedire e ricevere messaggi pacchettizzati.
- Lo standard *de facto* è **MPI (Message Passing Interface)**, che fornisce primitive per l'invio e ricezione (`send()`, `recv()`) e comunicazioni collettive (`broadcast`, `gather`).

Calcolo Accelerato (GPU) e Sistemi Ibridi

Oggi, l'enorme spinta dell'HPC deriva dall'uso delle **GPU**, schede video nate per la grafica ma perfette per il calcolo massicciamente *data-parallel*. NVIDIA ha introdotto **CUDA**, un modello in cui il codice sequenziale resta sulla CPU (host), la quale invia dati e delega il calcolo parallelo alla GPU tramite speciali funzioni dette **kernel**. Finito il calcolo, la GPU restituisce i dati alla CPU.

Nei supercomputer moderni, tutti questi modelli operano contemporaneamente nei cosiddetti **Sistemi Ibridi**. In un cluster moderno, utilizzerai MPI per far comunicare le macchine (nodi) attraverso la rete, OpenMP per parallelizzare il lavoro tra i core fisici della CPU dentro al nodo, e CUDA per scaricare i calcoli pesanti sulle GPU del singolo nodo.